

# 댓글 감성 분석

## 딥러닝(KcELECTRA) 기반 유튜브 댓글 감성 분류 시스템

인공지능 학습 결과서

*comment\_analysis.ipynb 기반 작성*

### 1. 개요

본 문서는 유튜버 위험도 분석 프로젝트의 평판 위험도 산출을 위해 구축한 댓글 감성 분류 시스템의 딥러닝 적용 과정을 기술합니다. YouTube 댓글 79,937건에 대해 사전학습된 한국어 트랜스포머 모델(KcELECTRA)을 활용하여 긍정/부정 감성을 자동 분류하고, 그 결과를 MySQL DB에 적재하는 전 과정을 다룹니다.

| 항목     | 내용                                                |
|--------|---------------------------------------------------|
| 분석 대상  | YouTube 댓글 79,937건 (채널 수준)                        |
| 사용 모델  | KcELECTRA-base-v2022 (beomi/KcELECTRA-base-v2022) |
| 분류 클래스 | 2클래스 — 긍정(1) / 부정(0)                              |
| 실행 환경  | Python 3.13, PyTorch, MPS(Apple Silicon GPU)      |
| 데이터베이스 | MySQL (youtube_model_db / comments_table)         |
| 출력     | sentiment_label (0/1) + sentiment_prob (확률값)      |

## 2. 전체 파이프라인

전체 딥러닝 추론 파이프라인은 아래 5단계로 구성됩니다.

| 단계     | 작업 내용                       | 주요 코드/함수                                                                         |
|--------|-----------------------------|----------------------------------------------------------------------------------|
| Step 0 | 라이브러리 로드 및 DB 연결 확인         | <code>db_utils.test_connection()</code>                                          |
| Step 1 | DB에서 댓글 데이터 로드              | <code>db_utils.run_query(SQL)</code>                                             |
| Step 2 | 데이터 전처리 (결측치 제거, 텍스트 추출)    | <code>dropna()</code> , <code>.tolist()</code>                                   |
| Step 3 | KcELECTRA 모델 로드 및 디바이스 설정   | <code>AutoTokenizer</code> ,<br><code>AutoModelForSequenceClassification</code>  |
| Step 4 | 배치 단위 추론 (토큰화 → 모델 → 확률 계산) | <code>tokenizer()</code> , <code>model(**inputs)</code> , <code>softmax()</code> |
| Step 5 | 결과 병합 및 DB 적재               | <code>comments_df</code> 병합 → UPDATE SQL                                         |

## 3. 환경 구성

### 3-1. 사용 라이브러리

| 라이브러리        | 버전/용도                                                                                                      |
|--------------|------------------------------------------------------------------------------------------------------------|
| torch        | PyTorch — 딥러닝 프레임워크, 텐서 연산 및 GPU 관리                                                                        |
| transformers | Hugging Face — 사전학습 모델 로드 ( <code>AutoTokenizer</code> , <code>AutoModelForSequenceClassification</code> ) |
| sqlalchemy   | Python ↔ MySQL 연결 및 SQL 실행 ( <code>text()</code> 함수 활용)                                                    |
| pymysql      | MySQL 드라이버                                                                                                 |
| dotenv       | DB 접속 정보 환경변수 관리 ( <code>.env</code> 파일)                                                                   |
| db_utils     | 자체 모듈 — DB 연결, 쿼리 실행, 결과 적재 유틸리티                                                                           |

### 3-2. 실행 디바이스

Apple Silicon(M 시리즈) 환경에서 MPS(Metal Performance Shaders) 백엔드를 활용하여 GPU 가속 추론

을 수행했습니다.

```
device = torch.device('mps' if torch.mps.is_available() else 'cpu')
```

MPS는 Apple Silicon Mac의 GPU를 PyTorch에서 사용하기 위한 백엔드입니다. CUDA(NVIDIA GPU)와 동일한 방식으로 .to(device) 메서드로 모델과 데이터를 디바이스에 할당합니다.

## 4. 데이터

### 4-1. 데이터 구성

MySQL DB의 comments\_table에서 text가 null이 아닌 댓글을 전체 조회하여 분석에 사용했습니다.

| 컬럼명                | 타입      | 설명                                |
|--------------------|---------|-----------------------------------|
| comment_id         | VARCHAR | 댓글 고유 ID (PK, UPDATE 쿼리의 조인 키)    |
| channel_identifier | VARCHAR | 채널 식별자 (CH0xxxx 형식)               |
| text               | TEXT    | 댓글 원문 — 모델 입력값                    |
| like_count         | INT     | 댓글 좋아요 수 (분석 미사용)                 |
| sentiment_label    | INT     | 감성 예측 결과 — 0: 부정, 1: 긍정 (추론 후 추가) |
| sentiment_prob     | FLOAT   | 긍정 클래스 확률값 0~1 (추론 후 추가)          |

### 4-2. 전처리

전처리는 최소화하여 사전학습 모델의 특성을 최대한 활용했습니다.

- text 컬럼이 null 인 행 제거 (dropna)
- comments\_df['text'].tolist() — 모델 입력 리스트 생성
- 특수문자, 이모지, 외국어(영어·일어·중국어) 포함 원문 그대로 입력 — KcELECTRA 가 다국어에 강건하므로 별도 필터링 미적용

실제 데이터에는 한국어 외에도 영어("I love you so much Aengie!!"), 일본어("今はTWICE"), 중국어("沒想到您對棒球也很熱情") 등 다국어 댓글이 혼재합니다. KcELECTRA는 한국어 특화 모델이므로 외국어 댓글

의 경우 예측 신뢰도가 낮을 수 있습니다.

## 5. 딥러닝 모델 — KcELECTRA

### 5-1. 모델 선택 배경

한국어 댓글 감성 분류에 특화된 사전학습 모델이 필요했으며, 다음 이유로 KcELECTRA-base-v2022를 선택했습니다.

- 한국어 코퍼스 및 댓글 데이터로 사전학습되어 구어체, 신조어, 이모티콘 처리에 강함
- BERT 대비 학습 효율이 높고 동일 파라미터 수에서 성능이 우수
- 감성 분류 태스크에 맞게 파인튜닝된 체크포인트 제공 (num\_labels=2)
- base 모델(768 차원)로 79,937 건을 배치 처리 시 현실적인 추론 시간 확보

### 5-2. 모델 아키텍처

출력된 모델 구조를 기반으로 KcELECTRA의 핵심 구성 요소는 다음과 같습니다.

| 구성 요소               | 세부 내용                                                  |
|---------------------|--------------------------------------------------------|
| Embedding           | 토큰 임베딩 + 포지션 임베딩 + 토큰 타입 임베딩 → LayerNorm → Dropout     |
| ELECTRA Encoder     | 12개 Transformer 레이어 (ElectraLayer × 12)                |
| Self-Attention      | 12 헤드, 768 차원 → Query/Key/Value 선형 변환 후 어텐션 → Dropout  |
| Feed-Forward        | 768 → 3072 (GELU 활성화) → 768 → LayerNorm → Dropout      |
| Classification Head | 768 → 768 (Dense, GELU) → Dropout → 768 → 2 (out_proj) |
| 출력 형태               | logits shape: [batch_size, 2] — 부정(0), 긍정(1) 클래스 로짓    |

### 5-3. 모델 로드

```
MODEL_NAME = "beomi/KcELECTRA-base-v2022"
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForSequenceClassification.from_pretrained(MODEL_NAME,
num_labels=2)
model.to(device)
```

```
model.eval() # 추론 모드 전환 (Dropout 비활성화, BatchNorm 고정)
```

*model.eval()은 Dropout 레이어를 비활성화하고 BatchNorm을 추론 모드로 전환합니다. 학습 시와 다른 동작을 방지하기 위해 추론 전 반드시 호출해야 합니다.*

## 6. 추론 (Inference)

### 6-1. 배치 처리 설계

79,937개 댓글을 한 번에 모델에 투입하면 메모리 오버플로우가 발생하므로 배치(batch) 단위로 나누어 처리했습니다.

| 설정         | 값       | 설명                                                  |
|------------|---------|-----------------------------------------------------|
| batch_size | 16      | 한 번에 모델에 입력하는 댓글 수. 메모리 상황에 따라 32~64로 늘릴 수 있음       |
| max_length | 128     | 토큰 최대 길이. 128을 초과하면 자름(truncation), 짧으면 패딩(padding) |
| 총 배치 수     | 약 4,997 | $79,937 \div 16 = 4,996.1 \rightarrow$ 올림           |

### 6-2. 추론 단계별 설명

#### ① 토큰화 (Tokenization)

댓글 텍스트를 모델이 이해할 수 있는 숫자 시퀀스(토큰 ID)로 변환합니다.

```
inputs = tokenizer(batch_texts, return_tensors='pt', padding=True,
truncation=True, max_length=128).to(device)
```

| 파라미터                | 설명                                              |
|---------------------|-------------------------------------------------|
| return_tensors='pt' | PyTorch 텐서 형태로 반환 (tf=TensorFlow, np=NumPy도 가능) |
| padding=True        | 배치 내 가장 긴 시퀀스 길이에 맞춰 짧은 것은 [PAD] 토큰으로 채움        |
| truncation=True     | max_length(128) 초과 시 잘라냄                        |
| .to(device)         | 토큰화 결과를 MPS(GPU) 디바이스로 이동                       |

## ② 순전파 (Forward Pass)

그라디언트 계산을 비활성화(torch.no\_grad())하여 메모리와 연산을 절약하면서 모델에 입력합니다.

```
with torch.no_grad():
    outputs = model(**inputs)
    logits = outputs.logits # shape: [batch_size, 2]
```

*torch.no\_grad()는 추론 시 역전파를 위한 중간 연산 그래프를 저장하지 않아 메모리를 절약합니다. 학습이 아닌 추론 단계에서는 항상 사용해야 합니다.*

## ③ 확률 변환 (Softmax)

모델이 출력한 로짓(logit)은 원시 점수로, Softmax를 거쳐 합이 1인 확률 분포로 변환합니다.

```
probs = torch.nn.functional.softmax(logits, dim=-1) # shape: [batch_size, 2]
preds = torch.argmax(probs, dim=-1) # 가장 높은 확률의 클래스 인덱스
probabilities.extend(probs[:, 1].cpu().numpy()) # 긍정 확률만 저장
```

## ④ 결과 수집

배치마다 예측 라벨과 긍정 확률을 리스트에 누적합니다.

```
predictions.extend(preds.cpu().numpy()) # 예측 라벨 (0 또는 1)
probabilities.extend(probs[:, 1].cpu().numpy()) # 긍정 확률
```

*.cpu().numpy()는 GPU 메모리에 있는 텐서를 CPU로 이동 후 NumPy 배열로 변환합니다. Python 리스트로 변환하여 일반 연산에 사용할 수 있게 합니다.*

# 7. 분석 결과

## 7-1. 출력 컬럼

| 컬럼              | 타입    | 설명                    | 예시       |
|-----------------|-------|-----------------------|----------|
| sentiment_label | int   | 예측 클래스 — 0: 부정, 1: 긍정 | 1        |
| sentiment_prob  | float | 긍정 클래스 확률 (0.0 ~ 1.0) | 0.521315 |

## 7-2. 결과 샘플

실제 추론 결과에서 발췌한 샘플 댓글입니다.

| 댓글 (text)                                                                                                         | label | prob  | 해석            |
|-------------------------------------------------------------------------------------------------------------------|-------|-------|---------------|
| 아쉽지만 누구보다 빛났던 무대  하루 님 정말 최고였습니다 | 1     | 0.526 | 긍정 — 응원 및 칭찬  |
| 점점 망해가네 하... 내 최애프로였는데           | 0     | 0.491 | 부정 — 아쉬움/불만   |
| 각 계원에대 꼭 와주세요 목 빠지게 기다려요!!                                                                                        | 1     | 0.511 | 긍정 — 기대/요청    |
| 에스컬레이터 속도 안 높일 거면 한줄서기해야지                                                                                         | 0     | 0.481 | 부정 — 비판/불만    |
| 형님 일주일만입니다. 기다리고 있었습니다                                                                                            | 0     | 0.487 | 경계값 — 중립에 가까움 |

감성 확률이 0.48~0.53 범위에 집중된 것을 볼 수 있습니다. KcELECTRA가 별도의 파인튜닝 없이 zero-shot으로 사용되었기 때문에 확률 분포가 0.5 부근에 몰리는 경향이 있습니다. 분류 성능 향상을 위해서는 라벨링된 유튜브 댓글 데이터셋으로 추가 파인튜닝을 권장합니다.

## 8. DB 적재 프로세스

추론 결과를 MySQL DB의 comments\_table에 적재하는 과정은 3단계로 설계되었습니다.

| 단계     | 작업                 | 세부 내용                                                                   |
|--------|--------------------|-------------------------------------------------------------------------|
| Step 1 | 컬럼 존재 여부 확인        | INFORMATION_SCHEMA.COLUMNS 조회 → sentiment_label 컬럼이 없으면 ALTER TABLE로 추가 |
| Step 2 | 임시 테이블 생성 및 데이터 삽입 | temp_comments_sentiment 임시 테이블에 분석 결과 INSERT (체크 단위 처리)                 |
| Step 3 | 실제 테이블 업데이트 및 정리   | comment_id를 키로 JOIN UPDATE 실행 → 임시 테이블 DROP                             |

### 8-1. 발생 오류 및 해결 방법

DB 적재 과정에서 두 가지 오류가 발생했습니다.

### 오류 1 — Collation 불일치 (OperationalError 1267)

```
OperationalError: (1267, "Illegal mix of collations (utf8mb4_unicode_ci,IMPLICIT) and (utf8mb4_0900_ai_ci,IMPLICIT) ")
```

원인: comments\_table과 임시 테이블의 Character Set Collation 설정이 달라 WHERE c.comment\_id = t.comment\_id 비교 연산에서 오류 발생

해결방법:

- 임시 테이블 생성 시 COLLATE utf8mb4\_unicode\_ci 명시
- 또는 MySQL 서버 기본 Collation 을 통일 (my.cnf 설정 변경)
- db\_utils.update\_sentiment\_results() 내부에서 CONVERT() 함수로 캐스팅하는 방법도 가능

### 오류 2 — chunksize 파라미터 미지원 (TypeError)

```
TypeError: update_sentiment_results() got an unexpected keyword argument 'chunksize'
```

원인: db\_utils.update\_sentiment\_results()가 chunksize 파라미터를 지원하지 않음

해결방법: db\_utils 모듈 내부에 청크 처리 로직을 추가하거나, 호출 코드에서 루프로 분할 처리

## 9. 한계 및 개선 방향

| 한계                             | 개선 방향                                                  |
|--------------------------------|--------------------------------------------------------|
| zero-shot 사용으로 확률값이 0.5 부근에 집중 | 유튜브 댓글 라벨링 데이터셋으로 파인튜닝 (2,000~5,000건 수작업 라벨링 후 학습)     |
| 2클래스(긍정/부정)만 분류 — 중립 클래스 없음    | 3클래스(긍정/중립/부정) 분류로 확장. 0.45~0.55 구간을 중립으로 후처리 가능       |
| 외국어 댓글 처리 품질 미검증               | multilingual-BERT 또는 XLM-RoBERTa로 대체하거나, 언어 감지 후 분리 처리 |
| DB Collation 오류로 자동 적재 미완성     | 임시 테이블 생성 시 Collation 명시 또는 MySQL 서버 설정 통일             |



| 한계                          | 개선 방향                             |
|-----------------------------|-----------------------------------|
| 배치 크기 16으로 79,937건 처리 시간 소요 | 배치 크기 확대(32~64) 또는 멀티프로세싱으로 병렬 처리 |

## 10. 위험도 산출 연계 방안

댓글 감성 분석 결과는 평판 위험도 산출에 다음과 같이 활용됩니다.

| 단계          | 처리 내용                                                        |
|-------------|--------------------------------------------------------------|
| 채널별 집계      | channel_identifier 기준으로 긍정 비율 / 부정 비율 / 평균 sentiment_prob 계산 |
| 감성 위험 점수 산출 | 부정 비율이 높을수록 점수 상승 → 0~1 정규화                                  |
| 평판 위험도 합산   | 감성 위험 점수(50%) + 민감 키워드 점수(50%) → 평판 위험도                      |
| 최종 위험도 통합   | 평판(33%) + 트래픽(33%) + 팬덤(33%) → ABCDF 등급 분류                   |